

Critical error rate in practice: context-aware variant significance for nanopore amplicon pipelines

Josh Walker

Abstract

The critical error rate (CER) framework [1] provides a per-variant significance statistic p^* that is interpretable against known platform error rates. The framework as originally posed assumes a single context-free error rate, which is appropriate for non-homopolymer substitutions in nanopore data but breaks down for homopolymer (HP) length variants, where empirical error rates vary by a factor of two or more across HP length and base context [2]. We present a context-aware extension of the framework, retaining the single binomial equation but replacing the uniform error rate p with a context-specific q_{ctx} derived from empirical measurement. To preserve cross-variant comparability under context-aware q_{ctx} , we introduce the *CER factor*, p^*/q_{ctx} , as the primary derived metric. The factor is dimensionless, comparable across variants and specimens, degrades linearly under q_{ctx} misspecification, and re-scales cleanly when the empirical error table is updated. We describe the implementation in the speconsense pipeline, including N-denominator choice for post-phasing validation, identity-group construction, pairwise CER comparison, and a metadata reproducibility design that allows stored output to be re-evaluated against revised assumptions without re-running the pipeline.

1 Introduction

The critical error rate (CER) framework, introduced in [1], asks the inverse of the conventional p-value question: rather than testing whether an observed variant is significant under an assumed error rate, it derives the per-position error rate p^* at which the observed pattern would become plausible as artifact. A practitioner then compares p^* against their platform’s known error rate; well-supported variants produce p^* values several times the platform rate, providing a transparent margin for judgment.

The framework’s strength is that p^* has the same units as the platform error rate the reader already knows, decoupling the analysis from the moving target of basecaller and chemistry improvements. A p^* of 6.8% is robust against any reasonable nanopore configuration without committing to a specific assumed rate at output time.

The implementation described here is deployed in speconsense [3], the consensus generation tool used in the high-throughput fungal barcoding workflow of Russell [4]. Empirical examples in this report draw on a sequencing run (ont98) produced by Mycota Lab under that workflow.

1.1 The context-dependence problem

The framework as originally formulated assumes a single per-position error rate p characterizes the platform. For non-homopolymer substitutions in current nanopore chemistry, this is approximately true: empirical post-clustering rates are tightly clustered around 0.6%–1.0% across position contexts [2]. The framework’s $p^* = 6.8\%$ headline against an assumed rate of 1.5% is robust to this spread.

For homopolymer-length variants, the assumption breaks down. Homopolymer (HP) length errors are the dominant systematic error mode in nanopore sequencing [5], and their empirical rates depend strongly on context:

- HP length error rate increases monotonically from $\sim 0.7\%$ per position at $L = 1$ to $\sim 1.5\%$ at $L = 5$, and steeply thereafter [2] (raw filtered measurement; shipped Phase-1 values under the re-estimated table are $\sim 0.47\text{--}0.87\%$ across the same range, Appendix B).
- G/C homopolymers consistently produce higher error rates than A/T at matched length, with the gap widening at longer lengths.
- Deletion-biased errors dominate insertion-biased errors at a 3–5:1 ratio across moderate HP lengths.
- Per-run and per-basecaller variation contribute additional spread of $\sim 0.6\times$ to $2.2\times$ on top of the context dependence.

A single assumed error rate cannot serve all of these contexts simultaneously without either over-suppressing variants in low-error contexts (e.g., $L = 1$ HP positions) or under-suppressing in high-error contexts (e.g., $L = 5$ G runs). The current speconsense default of 1.5% assumed error rate is calibrated to cover the non-HP substitution regime; applied uniformly to HP variants, it loses 30–50% of the available signal.

1.2 Contributions

This paper extends the CER framework to context-dependent error rates without abandoning its core appeal:

1. A **unified equation** that retains the single binomial structure of [1] but replaces L with a count of independent test sites n_{sites} and replaces q with a context-specific q_{ctx} . For multi-position variants ($K > 1$), the joint per-read error probability becomes $\prod_i q_{\text{ctx},i}$ instead of q^K , accommodating mixed contexts naturally.
2. A **practical decision metric**, the *CER factor*, defined as p^*/q_{ctx} . The factor is dimensionless and context-comparable by construction, recovering the cross-variant comparability that raw p^* loses under context-aware q_{ctx} . We compare it against alternative metrics (raw p^* , p-value) and recommend it as the primary annotation.
3. Implementation choices in the **speconsense pipeline** that follow from the framework: how N is chosen for post-phasing validation, how identity groups define the comparison set for pairwise CER, why phasing deliberately operates without statistical gating, how q_{ctx} is parameterized, and how HP detection is reordered relative to homopolymer normalization.
4. A **metadata reproducibility design** that stores enough information in each specimen’s output to recompute p^* , p-value, and CER factor under revised assumptions, without re-running the upstream pipeline.

This paper assumes the reader is familiar with the theoretical CER framework of [1] and the empirical HP error characterization of [2]. Section references to these companion documents are made where useful but the core results are not re-derived.

2 Unified equation

We retain the binomial structure of the CER paper [1]. The generalization replaces the amplicon length L with a count of independent test sites n_{sites} and replaces the single per-position error probability q with a context-specific q_{ctx} .

2.1 Single-position variants ($K = 1$)

For a candidate variant supported by M reads at a single position with context c in a specimen of N total reads,

$$n_{\text{sites}} \cdot P(\text{Binom}(N, q_{\text{ctx},c}) \geq M) = T, \quad (1)$$

where T is either an input significance level α (when solving for p^*) or an output p-value (when $q_{\text{ctx},c}$ is fixed and the equation is evaluated). This reduces to the formulation in [1] when $q_{\text{ctx},c} = q$ is uniform across positions and $n_{\text{sites}} = L$.

2.2 Multi-position variants ($K > 1$)

When a variant is distinguished from its comparison cluster at $K > 1$ positions, the joint per-read probability of erroring at all K positions under the null is the product of per-position rates:

$$\binom{n_{\text{sites}}}{K} \cdot P(\text{Binom}(N, \prod_{i=1}^K q_{\text{ctx},i}) \geq M) = T. \quad (2)$$

This generalization is essential for HP applications because mixed-context $K > 1$ variants are common: a multi-position variant may be distinguished at one HP position and one non-HP position simultaneously. Equation 2 reduces to $\binom{L}{K} \cdot P(\text{Binom}(N, q^K) \geq M) = T$ when all $q_{\text{ctx},i}$ are equal.

2.3 Bonferroni site counting

The site count n_{sites} is the number of independent positions tested. Three definitions are operationally distinct:

Per-base count. $n_{\text{sites}} = L$, the amplicon length in bases. This is the formulation in [1]; appropriate when every position is tested independently.

Per-site count with HP aggregation. $n_{\text{sites}} = (\text{non-HP positions}) + (\text{HP runs of } L \geq 2)$. Equivalently, n_{sites} is the length of the *homopolymer-compressed* consensus — the sequence obtained by collapsing each maximal run of identical bases to a single base (so CAAAGGGG has three sites: one per distinct run). A length-5 HP run is one site for length-variant testing, not five. For ITS amplicons of ~ 700 bp, this aggregation reduces n_{sites} modestly (typically by ~ 15 – 25%) because most HP runs are short.

Per-site count with alternative-base correction. Each non-HP position admits 3 alternative bases; each $L \geq 2$ HP run admits multiple alternative lengths. A strict Bonferroni would multiply n_{sites} by the number of plausible alternatives per site. Following [1], we treat the $q \rightarrow q/3$ uniform error split as implicit per-position correction over alternative bases for non-HP positions, and use empirical per-transition rates for HP positions (which are already specific to one alternative).

We adopt the per-site count with HP aggregation as the default. The choice affects p^* only logarithmically (a 25% reduction in n_{sites} shifts p^* by less than 5%).

3 The CER factor

3.1 Definition

The *CER factor* is the dimensionless ratio

$$\text{CER factor} = \frac{p^*}{q_{\text{ctx}}}. \quad (3)$$

It answers the question: *by what factor would the platform error rate at this variant’s context need to be inflated to make the observation plausible as artifact?* A factor of 4 means quadruple the empirical error rate would be needed; a factor of 1 corresponds exactly to the platform rate.

3.2 Comparison with alternative metrics

Three derived metrics summarize the same underlying CER surface:

Raw p^* . The original CER paper’s headline metric. Interpretable as “the per-position error rate that would explain this variant.” Comparable across variants only when q_{ctx} is approximately uniform.

p-value. The conventional statistic, computed by fixing q_{ctx} and evaluating Equation 1 for T . Universal 0–1 scale, composable for FDR and meta-analysis, exponentially sensitive to q_{ctx} misspecification.

CER factor. Defined by Equation 3. Dimensionless, context-independent on the output side, linearly sensitive to q_{ctx} misspecification, re-scalable when q_{ctx} updates.

Table 1 summarizes the trade-offs.

Table 1: Trade-off summary for derived CER metrics. “✓” indicates the metric is preferred on that dimension.

Property	Raw p^*	p-value	CER factor
Context-comparable across variants		✓	✓
Linear sensitivity to q_{ctx} error			✓
Composable for FDR / meta-analysis		✓	
Principled default decision threshold		✓	
Reinterpretable against future chemistry	✓		✓
Familiar to general statistical audience		✓	
Bounded dynamic range at strong end			✓

3.3 Why the factor is the right primary metric for amplicon pipelines

For speconsense’s primary use case — amplicon barcoding where consensus sequences are deposited into taxonomic reference databases — the CER factor’s properties align with what practitioners need:

1. **Cross-variant comparability is the dominant requirement.** Validators rank variants by confidence and apply a threshold. The factor provides a single comparable scale across HP and non-HP contexts.

2. **Linear sensitivity to q_{ctx} is honest.** Empirical error rates vary $\sim 0.6\times$ between sequencing runs on the same basecaller [2]. P-values would swing by orders of magnitude under this drift; the factor moves linearly, which more accurately reflects our uncertainty about the true error rate at any given site.
3. **Re-scaling under updated chemistry is direct.** If a future q_{ctx} table revises the rate at a context by a factor r , stored factors at that context can be rescaled by $1/r$ without recomputing from raw N and M . P-values must be recomputed from raw inputs; raw p^* does not require rescaling but loses cross-context comparability. A worked example of a full-table refresh and its downstream effects is given in Section 7.1.
4. **The robustness interpretation survives.** A factor of 4 is “quadruple the empirical floor would be needed to explain this as artifact” — a direct generalization of the raw p^* interpretation, retaining the human-interpretable units that motivated CER in the first place.

3.4 Default decision threshold

We propose a default factor threshold of ≥ 2 for accepting a variant: the observed pattern requires at least double the empirical context-specific error rate to be explained as artifact. The threshold is conservative under typical q_{ctx} uncertainty (run-to-run variation, basecaller drift) and recovers the boundary behavior of the existing default $p^* \geq 1.5\%$ against a non-HP $q_{\text{ctx}} \approx 0.7\%$ (factor ≈ 2.1).

The threshold is a tunable: stricter calls (factor ≥ 3) are appropriate for novel reference deposits; permissive calls (factor ≥ 1) are appropriate for diversity surveys where sensitivity matters more than specificity. Section 6 describes how the factor can be recomputed at any threshold from stored metadata without re-running the pipeline.

In the initial deployment the production default is factor ≥ 1.0 , not the factor ≥ 2 analyzed above. This is a pragmatic interim choice: the factor = 1 boundary is the straightforward “observation explainable by the empirical rate alone” cut, easy to reason about while the pipeline accumulates validation data. The threshold is expected to be tuned upward (likely toward factor ≥ 2 for deposit-quality calls) once the empirical distribution of factor values on real workloads is characterized. Because the factor is recorded per-variant in the metadata JSON (Section 6), threshold tuning does not require re-running the upstream pipeline.

3.5 FASTA annotation scheme

A single CER field is emitted per consensus header; the forensic detail lives in the per-specimen metadata JSON (Section 6).

```
>sample-1.v2 size=200 ric=100 rid=95.2 cer_factor=4.2
```

The header carries only **cer_factor**: a single dimensionless number, the primary decision metric; higher means more robust. Per-position detail (p^* , K , context tags, q_{ctx} values, reference cluster index) lives in the metadata JSON.

An earlier draft of this framework emitted a parallel **cer_details** field in the header encoding p^* , K , and context/ q pairs joined by **+**. We abandoned this in implementation: at $K > 1$ the field becomes a multi-token string that crowds the fixed-width **-all.fasta** header, inflates line length past comfortable terminal display, and duplicates data the metadata JSON already carries in structured form. The header is the triage surface (*does this variant pass?*); the JSON is the

forensic surface (*why does this variant pass?*). Splitting by role keeps the header scannable and the forensic detail queryable.

4 Parameterizing q_{ctx}

The unified framework demands a context classifier and a q_{ctx} table. This section describes the context taxonomy, the source of empirical values, and the operational choices for keeping q_{ctx} current.

4.1 Context classes

Each MSA position falls into one of a small set of context classes. The classification is deterministic from the consensus sequence at that position and depends on no upstream call:

Non-HP substitution. The consensus base is not part of an HP run of length ≥ 2 , and the variant differs by a substitution. Empirical q_{ctx} corresponds to the per-position substitution rate; for Dorado v5.0 on R10.4.1, $q_{\text{ctx}} \approx 0.0059$ [2].

HP-interface substitution. The consensus base is the first or last base of an HP run of length ≥ 2 , and the variant is a substitution. Empirical analysis [2] §4 demonstrates that interface substitutions occur at rates indistinguishable from non-HP substitutions; they are assigned the non-HP substitution rate.

HP length change at L . The variant differs from the consensus by a length change in an HP run of reference length $L \in \{2, 3, 4, 5\}$. Empirical q_{ctx} is read from the per- L table in [2] §3.3, pooled across base and direction (Section 4.1.1).

HP length change at $L \geq 6$. Suppressed by blanket normalization; no q_{ctx} assigned and no CER evaluation. This follows the recommendation of [2] §8.1: at $L \geq 6$ accuracy degrades sharply and variant calling becomes unreliable.

Indel at non-HP position. Treated as a substitution-class event with the non-HP rate as a placeholder. Indels at non-HP positions are rare in amplicon data and do not warrant a separate empirical column at this stage.

The classifier returns a context tag and a q_{ctx} value. The context tag records the base and direction of an HP-length event for output annotation (e.g., **hp-13-A-del1**), even though the q_{ctx} lookup uses only the length. For mixed-context $K > 1$ variants, the per-position tags are recorded as a list and the joint q_{ctx} is the product (Equation 2).

4.1.1 Pooling base and direction in HP q_{ctx}

The HP length-change class could in principle split along three sub-dimensions: base $b \in \{A, T, C, G\}$, reference length L , and direction/magnitude δ (deletion vs. insertion, one-base vs. multi-base). A fully specified table would require $\sim 4 \times 4 \times \geq 2 = 32+$ entries for the $L \leq 5$ regime alone. The shipped tables collapse b and δ , keeping only L : five HP entries plus two non-HP entries. This is a deliberate simplification driven by empirical signal-to-noise in [2]:

- **Base (pooled).** Per-base variation at $L \leq 5$ on Dorado v5.0 is small: maximum over minimum across bases is 1.14 at $L = 1$ rising to 1.39 at $L = 5$ [2] §7.6. The “worst-case base”

is not consistent across lengths (T at $L = 2-3$, G at $L = 4-5$), so no stable A/T-vs-C/G split is supported in this regime.

- **Direction (pooled).** The deletion-to-insertion ratio is 3–5:1 at moderate lengths [2] §3.4, but per-cell counts for insertions — especially at $(b, L, \delta = +1)$ — are sparse. Splitting would produce unstable rates for the less-populated cells.
- **Length (kept).** The per- L effect is the largest and most stable: rates rise monotonically from 0.67% at $L = 1$ to 1.13% at $L = 5$ on ont98 [2] Table 4, a $\sim 1.7\times$ span resolved above cross-run noise.

Run-to-run uncertainty on a single basecaller is plausibly comparable in magnitude to the $\sim 1.4\times$ per-base and $\sim 3-5\times$ direction effects, but is not characterized in the present empirical data [2]; for the insertion direction in particular, sparse per-cell counts in pooled data make finer segmentation unstable. The shipped scheme therefore captures length as the primary axis and pools the rest. Finer segmentation by base and direction is straightforward to add (extend the YAML with `hp-1{L}-{b}-{δ}` keys; the lookup function already accepts arbitrary string keys) if future data shows the finer cells stabilizing above the cross-run noise floor. The output context tag records the base and direction regardless, so downstream re-analysis against a finer table requires no upstream re-run.

4.2 Sources of q_{ctx} values

Three operational regimes for q_{ctx} are appropriate at different levels of pipeline maturity:

Phase 1 (initial deployment): shipped per-basecaller table. A static q_{ctx} table is bundled with `speconsense`, parameterized by basecaller identifier (e.g., `dorado-v5.0`). Values are drawn from the empirical analysis in [2], pooled across base and direction as described in Section 4.1.1. The table identifier is recorded in metadata (Section 6). Users select their basecaller via a CLI flag (`--error-model dorado-v5.0`) or accept the default.

Phase 2 (mature deployment): per-run estimation. q_{ctx} is estimated from the current run’s MSAs at pipeline startup. The estimator pools positions across all specimens after applying the outlier-and-bimodal-detection biological-variant filter of [2] §5.3. The estimated q_{ctx} is recorded in metadata so that downstream interpretation can use the run-specific values or fall back to the shipped defaults.

Override: user-supplied model. The `--error-model` flag accepts either a shipped model name, a user model in `~/config/speconsense/error_models/`, or a filesystem path to a YAML file with a `rates` mapping. Useful for research applications that need to characterize variant calls under synthetic or extreme error assumptions.

The first deployment supports Phase 1 only. Phase 2 is a substantial implementation effort and is left for future work; it is mentioned here so that the metadata schema (Section 6) is forward-compatible.

4.3 Stability across runs

Run-to-run variation within a single basecaller version is plausible (driven by flow cell quality, library preparation, and basecaller calibration drift) but is not characterized in [2]; the shipped Phase

1 table is derived from a single ont98 run on Dorado v5.0. Real-world deployment will encounter runs whose actual rates differ from the shipped values; the CER factor’s linear sensitivity to q_{ctx} means that any such divergence shifts factor values linearly rather than by orders of magnitude as p-values would under the same misspecification. Phase 2 per-run estimation would eliminate this source of uncertainty entirely.

5 Implementation in speconsense

This section describes how the unified framework integrates with the existing speconsense pipeline. The current implementation [3] performs recursive MSA-based variant phasing, post-phasing identity grouping, and pairwise CER validation. The context-aware framework requires changes at each of these stages.

5.1 N denominator choice

CER is computed only at post-phasing pairwise validation (Section 5.5); phasing itself uses local frequency and count thresholds without statistical gating, and any pass/fail decision from `cer_factor` is deferred to summarize (Section 5.4). The denominator N in the binomial therefore has a single definition at the only point where it enters the computation.

Identity-group sum. When validating one cluster against another within an identity group, N is the sum of read counts within the group, not the total specimen count. The pairwise comparison’s null hypothesis is “these two clusters represent the same underlying sequence, with the smaller cluster arising as artifact of the larger”—a question that operates on the population of reads sharing the same identity-group membership, not on the broader specimen pool.

In the current implementation, this is computed in `_validate_identity_group` as `group_N = sum(len(subclusters[i]['read.ids'])) for i in sorted_indices)`.

Why the identity-group sum is the right choice. The artifact hypothesis under test is “the candidate cluster is misclassified reads from its sibling.” The relevant population is the union of reads that could plausibly have ended up in either cluster — the identity group. Using the total specimen reads as N would inflate the denominator relative to the hypothesis under test, producing artificially small p^* values (and CER factors) and over-accepting weak variants. Using only the candidate cluster’s own size as N would conversely understate the denominator and over-reject. The identity-group sum corresponds exactly to the population over which the candidate is hypothesized to be a redistribution of reads from validated siblings.

Why not “total specimen reads”. A natural alternative is to use the total specimen input reads as N , on the conservative reasoning that any read in the specimen could in principle have appeared in this position. We considered and rejected this. The artifact null is not “any specimen read could have erred here” — which is what total specimen N models — but “among the reads belonging to this identity group, could redistribution have produced the candidate’s count.” Reads outside the group are part of unrelated identity groups and do not enter the artifact mechanism for this candidate.

5.2 Identity groups

An *identity group* is a set of clusters whose pairwise HP-normalized identity exceeds a threshold for *every* pair in the group. Groups define the comparison set for pairwise CER: a candidate variant is tested against members of its own identity group, not against the entire specimen.

Construction. Groups are formed by complete-linkage hierarchical clustering (`scipy.cluster.hierarchy`) on pairwise HP-normalized adjusted-identity distance, cut at $1 - \text{group_identity}$. The default threshold is 85% identity (15% adjusted-identity distance), tunable via `--group-identity`. Under complete linkage, every pair of members in a group satisfies the identity threshold; under single-linkage union-find, only a single connecting chain is required.

Why complete linkage over union-find. We adopted complete linkage after observing transitive-chain failures on environmental-DNA specimens. eDNA samples commonly contain taxonomically distinct but related sequences (e.g., sibling species amplified together) that would form a single chain under union-find at any reasonable threshold, collapsing real organismal diversity into one identity group and producing misleading CER comparisons between unrelated amplicons. Complete linkage ensures that grouping survives the “furthest pair” test: two clusters whose pairwise identity is below threshold cannot end up in the same group via an intermediate. This matches the within-specimen identity criterion we want the group to enforce.

Role in CER decisions. Within each identity group, clusters are ordered by size. The largest cluster (the *anchor*) auto-passes CER without comparison. Each smaller cluster is tested pairwise against all already-validated clusters in the group; the candidate’s CER factor is the *minimum* across all pairwise comparisons, corresponding to the “nearest plausible artifact source.” Acceptance requires the minimum factor to exceed the decision threshold.

Why groups, not any-to-any. The artifact hypothesis is that a small cluster represents reads misclassified from a similar cluster. This hypothesis only makes sense within a similarity group; clusters of distinct sequences cannot serve as artifact sources for each other. Restricting the comparison set to identity-group members both encodes this hypothesis and reduces compute by avoiding $O(n^2)$ comparisons across the specimen.

5.3 Pairwise CER comparison

Pairwise CER between candidate cluster A and validated reference cluster B proceeds as follows:

1. Compute pairwise differences $K = \text{count_variant_differences}(A_{\text{cons}}, B_{\text{cons}})$, where each substitution contributes 1 and each contiguous indel event contributes 1 (regardless of indel length).
2. For each of the K differing positions, classify the context and look up $q_{\text{ctx},i}$.
3. Compute the joint per-read error probability $\prod_i q_{\text{ctx},i}$.
4. Compute n_{sites} over the longer of the two consensus by counting maximal base runs (Section 2.3): each non-HP position contributes one site, and each HP run of length ≥ 2 also contributes one site regardless of run length.
5. Take N as the identity-group sum (Section 5.1) and M as the candidate cluster’s read count.

6. Solve Equation 2 for p^* at the configured α , then compute the CER factor.

The candidate’s reported CER factor is the minimum over all pairwise comparisons against validated cluster references. This conservative choice reflects the worst-case artifact source and ensures that if any nearby validated cluster could plausibly explain the candidate as artifact, the candidate is suppressed.

5.4 Annotation in core, gating in summarize

The speconsense pipeline partitions responsibility across two executables: `speconsense` (core) produces consensus sequences with per-variant CER annotations, and `speconsense-summarize` consumes the annotated output and applies user-visible thresholds. CER gating lives exclusively in summarize.

Every non-anchor candidate is pairwise-compared against all larger peers in its identity group, annotated with a `cer_factor` (per-position multiplicative inflation; worst-case across peers), and flows through to the FASTA output. Records with `cer_factor` below the user-configured `--min-cer-factor` are routed by summarize to a separate `.ns` (non-significant) output file, mirroring the primary output layout; records with `cer_factor` undefined (anchors, comparisons that failed to converge) always pass.

This separation is deliberate. CER gating answers a question with a tunable threshold: which variants should a reader *see* as primary calls? That threshold depends on the reader’s application (reference deposit vs. diversity survey), their platform’s current error characterization, and the significance level chosen for the run. Baking the gate into core would force re-running clustering and consensus generation to retune the threshold. Keeping annotation in core and gating in summarize makes threshold retuning a seconds-scale rerun of summarize on the same FASTA, and preserves the sub-threshold clusters as inspection artifacts rather than destroying them.

5.5 Phasing without statistical gating

Recursive MSA-based phasing in speconsense produces candidate clusters using only local frequency and count thresholds (`--min-variant-frequency`, `--min-variant-count`); CER is not evaluated during phasing. This is a deliberate design choice, not an oversight, and we intend to keep it that way under the context-aware framework.

Phasing as candidate generation. The role of phasing is to *partition* reads into candidate haplotype clusters based on positional allele patterns. Phasing answers the structural question “which reads co-vary at which positions,” producing a set of candidate clusters whose existence is a hypothesis to be tested downstream. The role of CER is to *evaluate* those candidates against the artifact null. Conflating generation and evaluation forces every phasing decision through the CER machinery, which is both expensive and structurally inappropriate: at the moment a phasing split is being considered, the candidate cluster has not yet had its consensus generated, so its K relative to siblings is unknown and the appropriate q_{ctx} values cannot be computed.

Why post-phasing CER is preferred. Three reasons:

1. **Mixed-context $K > 1$ requires consensus alignment.** The unified equation requires per-position $q_{\text{ctx},i}$ values for each of the K positions distinguishing the candidate from its sibling.

These are known only after both consensus are generated and aligned, which happens post-phasing. A phasing-time gate would be limited to $K = 1$ evaluation against an arbitrary reference, losing the substantial sensitivity gains of multi-position evaluation (Section 2).

2. **Phasing’s local thresholds are fast and effective pre-filters.** The existing `--min-variant-count 5` and `--min-variant-frequency 0.10` thresholds eliminate the overwhelmingly majority of trivial splits (a single-read singleton at a random position) without any statistical machinery. CER’s contribution is discriminating among the remaining candidates, all of which have at least modest local support. Running CER at every phasing step would be largely wasted work.
3. **Identity-group N depends on phasing’s output.** The identity-group sum used as N in pairwise CER (Section 5.1) is determined by the set of clusters phasing produces and how they group by similarity. The hypothesis a phasing-time gate would test (“is this sub-split significant against *some* pool”) has no well-defined N until the full set of clusters is known.
4. **Cleaner primary consensus via noise excision.** Indiscriminate phasing followed by post-validation rejection lets non-significant variants be cleanly identified as their own *ns* (not significant) clusters rather than being absorbed back into a parent and manifesting as IUPAC ambiguity codes. With a phasing-time gate, reads that would have formed a non-significant split remain in their parent cluster, contributing to ambiguity at the affected positions and dirtying the parent’s consensus. The no-gate design separates “structurally a candidate variant” from “statistically meaningful,” so a primary consensus accumulates ambiguity only from positions where reads genuinely could not be phased apart at all—not from positions where phasing succeeded but the resulting cluster was too weakly supported to report. The non-significant reads remain visible in the output as *ns* clusters, available for inspection without polluting the significant consensus.

What about over-splitting? A reasonable concern is that indiscriminate phasing produces clusters that should never have been split, inflating downstream compute and adding noise. In practice, the local frequency and count thresholds do most of this work: a 10% / 5-read floor prevents single-read splits and cleanly catches obvious noise. Post-phasing identity-group construction (Section 5.2) collapses near-identical splits via union-find, and pairwise CER then suppresses weakly-supported candidates. The pipeline’s three-stage structure (local-threshold phasing → similarity-based grouping → statistical validation) cleanly separates concerns and matches each stage to the question it is structurally suited to answer.

5.6 HP detection and normalization

The pipeline treats HP-length variation at three distinct stages with deliberately different normalization defaults. Taken together, these stages admit HP length variants at $L \leq 5$ into CER evaluation with an appropriate q_{ctx} while suppressing the noisier $L \geq 6$ regime, as motivated by [2] §8.1.

Stage 1 — initial clustering identity. Initial read clustering (both the MCL K-NN graph and the greedy algorithm) uses *raw* edlib identity, $1 - \text{editDistance} / \max(|s_1|, |s_2|)$, computed in `SpecimenClusterer.calculate.similarity`. No homopolymer normalization is applied at this stage. The threshold `--min-identity` (default 0.90) filters raw pairs. HP length variants at $L \leq 5$ therefore do not drive separation at this stage for the typical amplicon: a single-base HP- length

disagreement on a ~ 700 bp ITS read contributes $\sim 0.15\%$ to raw distance, well inside the 10% tolerance of the default threshold, so reads with HP length polymorphism pool into the same initial cluster in practice. This is a threshold-driven outcome, not an explicit HP normalization; sequences with accumulated error that crosses the threshold can still separate.

Stage 2 — phasing variant detection. MSA-based variant detection (`is_variant_position_with_composition` in `speconsense.msa`) subtracts the HP-extension contribution from the raw base-composition counts before thresholding, but does so only for HP runs whose length meets the `min_hp_length` threshold of the HP-context builder (default 6). At $L \leq 5$ the HP-extension composition is empty, so the subtraction is a no-op and HP length variants are surfaced as candidate phasing positions. At $L \geq 6$ the subtraction zeros the variant count and the position is suppressed. Phasing therefore *does* split an initial cluster when its reads disagree at an $L \leq 5$ HP-length column, producing separate consensus clusters that flow downstream.

Stage 3 — post-phasing HP-equivalence merging. Clusters whose consensus differ only in HP lengths are candidates for merging via `are_homopolymer_equivalent`, again with `min_hp_length` of 6. Two clusters differing only at an $L \leq 5$ HP position are not merged and are carried forward as distinct variants; differences at $L \geq 6$ are treated as HP-equivalent and collapsed.

The three stages together enforce the following regime for HP length variation. At $L \leq 5$: Stage 1 pools reads through threshold tolerance (raw single-base indels are below the 10% distance budget), Stage 2 surfaces the column as a phasing candidate, and Stage 3 leaves the resulting split intact. At $L \geq 6$: Stage 1 still pools most reads through threshold tolerance, Stage 2 suppresses the variant column entirely, and Stage 3 collapses any residual split via HP-equivalence merging. The `min_hp_length` defaults at Stages 2 and 3 are intentionally aligned to 6: lowering it at Stage 2 would let high-error long runs drive spurious splits, and lowering it at Stage 3 would erase the short-HP variants that Stage 2 just surfaced.

Downstream handling. Surviving $L \leq 5$ HP-length variants reach pairwise CER evaluation as $K = 1$ candidates whose single differing position carries an `hp-1{L}-{base}-{dir}{count}` context tag (e.g., `hp-13-A-del1`; Section 4). The context classifier’s `hp_min_length` is set to 2 so that any run of two or more bases contributes an HP tag to pairwise comparisons; the $L \leq 5$ vs $L \geq 6$ question has already been settled by Stages 2 and 3, so the classifier does not need to re-enforce it.

Ambiguity handling. The pipeline does not currently absorb sub-threshold HP length variants into the parent consensus as explicit HP-ambiguity annotations; variants that fail CER in summarize are routed to the `.ns` output (Section 5.4) and the primary consensus retains its own modal HP length. Encoding HP length variation as a structured header annotation (e.g., `poly_A[5..6]`) for downstream inspection remains an open design question.

6 Metadata reproducibility

The CER computation depends on several inputs — N , M , K , n_{sites} , q_{ctx} , α — and a stable mapping from position to context. As empirical q_{ctx} values are revised (new basecaller versions, per-run estimation, improved error characterization), practitioners should be able to recompute CER factor and p-value from *stored* metadata, without re-running the pipeline. This section specifies the metadata schema that supports this.

6.1 Specimen metadata JSON

Each specimen produces a metadata JSON file at the standard output location. The current schema [3] contains parameters, input information, and primer settings; we extend it with CER reproduction data:

```
{
  "schema_version": "2.0",
  "version": "0.7.7",
  "sample_name": "ONT08.75-C10--iNat262724197",
  "timestamp": "2026-04-20T18:09:14.624236",
  "parameters": {
    "algorithm": "graph",
    "min_identity": 0.9,
    "min_size": 3,
    "significance_level": 1e-5,
    "group_identity": 0.85,
    "error_model": "dorado-v5.0",
    ...
  },
  "total_input_reads": 1000,
  "identity_groups": [
    {
      "group_id": "g1",
      "group_N": 974,
      "members": ["1.v1", "1.v2", "1.v3", ...]
    }
  ],
  "variants": [
    {
      "cluster_id": "1.v1",
      "identity_group": "g1",
      "group_rank": 1,
      "variant_rank": 1,
      "M": 462, "N": 974,
      "K": null, "n_sites": null, "context_tags": null,
      "q_ctx_per_position": null,
      "compared_against_idx": null,
      "cer_factor": null, "cer_pstar": null,
      "err_factor": 0.303,
      "err_factor_obs_sum": 3.13,
      "err_factor_exp_sum": 10.33,
      "err_factor_cols": 781
    },
    {
      "cluster_id": "1.v3",
      "identity_group": "g1",
      "group_rank": 1, "variant_rank": 3,
      "M": 49, "N": 974, "K": 1, "n_sites": 561,
      "context_tags": ["hp-13-A-del1"],
      "q_ctx_per_position": [0.0097],
      "compared_against_idx": 0,
      "cer_factor": 2.085,
      "cer_pstar": 0.0202,
    }
  ]
}
```

```

    "err_factor": 0.453,
    "err_factor_obs_sum": 4.67,
    "err_factor_exp_sum": 10.32,
    "err_factor_cols": 780
  }
]
}

```

Cluster identifiers use the `{group_rank}.v{variant_rank}` form assigned by core (e.g., `1.v3` is the third-largest variant within the largest identity group). The anchor of each identity group is the entry with `compared_against_idx = null`; all CER fields for anchors are null by construction. Non-anchor variants carry the integer index (into the variant list) of the reference cluster against which `cer_factor` was computed. Fields `err_factor`, `err_factor_obs_sum`, `err_factor_exp_sum`, and `err_factor_cols` record the cluster homogeneity statistic described in Section 7 along with its raw components; downstream tools can recompute the factor under a revised q_{ctx} table without re-running the pipeline.

Derived scalars that are cheap to recompute from the stored inputs — $q_{\text{ctx,joint}} = \prod_i q_{\text{ctx},i}$ and the p-value at the stored q_{ctx} — are not persisted.

6.2 Recomputation workflow

Given a stored metadata JSON and a revised q_{ctx} table or significance level, recomputation proceeds:

1. For each variant, look up the new $q_{\text{ctx},i}$ values for the recorded `context_tags`.
2. Compute the new joint $q_{\text{ctx,joint}}^{\text{new}} = \prod_i q_{\text{ctx},i}^{\text{new}}$.
3. Recompute p^* from $(N, M, n_{\text{sites}}, K, \alpha^{\text{new}})$.
4. The CER factor becomes $p^*/q_{\text{ctx,joint}}^{\text{new}}$ (or, for cross-variant reporting on a per-position scale, $(p^*)^{1/K}/\bar{q}_{\text{ctx}}^{\text{new}}$ where $\bar{q}_{\text{ctx}}^{\text{new}}$ is the geometric mean of the new per-position rates; the implementation currently reports the joint form).
5. If a p-value is needed, compute $n_{\text{sites}} \cdot P(\text{Binom}(N, q_{\text{ctx,joint}}^{\text{new}}) \geq M)$.

The original N , M , K , context tags, and consensus-derived n_{sites} are immutable properties of the sequencing data; no upstream pipeline run is needed to revise the significance metric.

6.3 Schema versioning

The `schema_version` field allows downstream tools to detect missing fields and degrade gracefully. Version 1.0 is the pre-context-aware schema; version 2.0 introduces the CER fields in this paper. Tools reading version 1.0 output can fall back to the original raw- p^* interpretation; tools reading version 2.0 output have full reproducibility.

6.4 Why this is worth the bytes

The metadata addition adds ~ 200 – 500 bytes per variant. For a typical ITS amplicon study with 1000 specimens and an average of 3 variants each, the total overhead is on the order of 1–2 MB across the run. This is negligible relative to the FASTA and FASTQ output. The benefit — re-computation under revised assumptions without re-running — is substantial: as basecaller versions evolve and empirical error tables improve, every stored output remains scientifically valid without recomputation cost.

7 Cluster denoising precedes CER

CER’s significance is sharply sensitive to K , the number of positions distinguishing the candidate from its reference peer. The joint per-read error probability is $\prod_i q_{\text{ctx},i}$, so a spurious additional differing position reduces $q_{\text{ctx},\text{joint}}$ by roughly a factor of q_{ctx} ($\sim 1/100$ at non-HP contexts) and inflates `cer_factor` accordingly. A noisy cluster whose consensus carries a handful of erroneous positions will therefore compute a dramatically inflated factor against its true peer, obscuring the distinction between real variants and cluster artifacts.

This places a specific demand on the upstream pipeline: CER is a meaningful discrimination metric only for clusters whose consensus is a faithful representation of the reads that compose it. Before CER evaluation, `speconsense` applies several denoising mechanisms whose purpose is to keep K honest:

- **Outlier removal during final consensus generation.** MAD-based detection of read-identity outliers excludes single-read noise that would otherwise inflate consensus-level disagreement at isolated positions.
- **Homopolymer-equivalence merging of near-identical post-phasing clusters** (pre-merge step), collapsing splits whose only distinguishing feature is HP length variation — the dominant systematic error mode that would inflate K by one or more positions per HP run.
- **MSA-based variant merging in summarize with IUPAC consensus**, further collapsing post-phasing clusters whose remaining differences are dominated by positions already classified as ambiguity-compatible under the specimen’s error profile.
- **A cluster-wide homogeneity statistic, `err_factor`**, reported per cluster and filtered in `summarize` via `--max-err-factor` (default 1.5). `err_factor` is the ratio of observed per-column disagreement to the q_{ctx} -predicted disagreement rate, pooled across the cluster’s MSA (non-gap consensus columns). Values near 1.0 indicate a cluster consistent with per-position basecaller noise; values well above 1.0 indicate internal heterogeneity beyond what the platform’s error model predicts. Unlike `cer_factor`, `err_factor` is peer-independent: it distinguishes novel-but-real sequences (low `err_factor`, potentially high `cer_factor`) from noise aggregates (high `err_factor`, often also spuriously high `cer_factor`). Filtering on `err_factor` before CER evaluation removes clusters whose K measurement would be unreliable regardless of the pairwise comparison. The raw components (`err_factor_obs_sum`, `err_factor_exp_sum`, `err_factor_cols`) are persisted in the metadata JSON (Section 6) so the factor can be recomputed under a revised q_{ctx} table.

None of these mechanisms replaces CER; they make CER’s K input trustworthy. The natural place in the pipeline for this preprocessing is between phasing and pairwise CER validation, which is where `speconsense` applies them.

7.1 Filter behavior under q_{ctx} refresh

Both `cer_factor` and `err_factor` are ratios with q_{ctx} (or a product of q_{ctx} values) in the denominator. A scalar revision of the q_{ctx} table by factor k therefore rescales both metrics by $1/k$: stored `cer_factor` values move as $p^*/q_{\text{ctx}}^{\text{new}}$, and stored `err_factor` values move as $\text{obs}/\text{exp}^{\text{new}}$. Because the two filters operate against different thresholds (`--min-cer-factor`, default 1.0; `--max-err-factor`, default 1.5) and `.1q` takes precedence over `.ns` when both filters fire, a q_{ctx} decrease (new values lower than old) produces a characteristic flow: records previously in `.ns` whose `err_factor` now

crosses 1.5 migrate to `.1q`, and a small tail of primary anchors whose `err_factor` crosses 1.5 migrate into `.1q` as well. The symmetric case (a q_{ctx} increase) produces the reverse flow.

A concrete example: refreshing the Dorado v5.0 table (Appendix B) from its initial values to the re-estimated values in [2] §8.6 is a $\sim 0.66\times$ rate change at non-HP substitution (and similar at other contexts). Re-running `speconsense-summarize` on a 849-specimen ont98 benchmark moved exactly 711 variants from `.ns` to `.1q` with zero movement in the reverse direction, and concurrently lost 43 variants (32 organisms) from the primary output. The 711-record flow is the expected conservation signature — same records, different classification. The 43-variant primary loss is the user-visible effect, concentrated in sole-variant anchors (`cer_factor` is `null` for anchors, so the user-visible filter for anchors is `--max-err-factor` alone) whose `err_factor` crossed the 1.5 threshold. One specimen was lost entirely (0.12%).

The metadata reproducibility design (Section 6) makes this recomputation cheap: stored N , M , K , context tags, and `err_factor` raw components recompute both ratios under the new table without re-running upstream core.

8 Validation

This section is a placeholder. Empirical content will be added as Phase 1 output accumulates.

Schema-2.0 metadata is now emitted by the current implementation. A full ONT98 run (955 specimens) at `dorado-v5.0` is available as initial input to the validation work below.

The planned validation reproduces the structure of the original CER validation [1] §8 but with the context-aware framework:

- Apply context-aware CER to the ONT98 dataset (955 specimens) and compare per-variant decisions against the existing raw- p^* implementation at default settings.
- Distribution of `cer_factor` on the 955-specimen workload, partitioned by variant class ($K=1$ non-HP, $K=1$ HP, $K>1$ mixed), to empirically motivate the `--min-cer-factor` default (Section 3).
- Sensitivity analysis: vary q_{ctx} by $\pm 50\%$ at each context class and quantify the fraction of decisions that flip. This characterizes the framework’s robustness to q_{ctx} misspecification under realistic uncertainty.
- HP-variant rescue: identify HP length variants that the current pipeline discards via blanket normalization but the context-aware framework would accept. Manual review of a sample to assess biological plausibility.
- Specimen-level outcomes: confirm that no specimens are lost (zero organism dropouts), consistent with the original CER validation.

Expected results: a moderate increase in HP-variant calls (mostly $L = 1\text{--}3$ variants currently discarded), a modest reduction in marginal substitution variants at HP-interface positions (where the empirical q_{ctx} is slightly lower than the current uniform assumption), and stable specimen-level counts.

9 Limitations

The framework as presented inherits the limitations of the underlying CER paper [1] and adds a few specific to the context-aware extension.

Cross-read error correlation. The binomial null assumes independence of errors across reads. Reads sharing the same PCR template can exhibit correlated errors (PCR-introduced substitutions, amplification bias); these are physically present in the library and not addressed by the CER framework, which establishes physical library presence rather than biological reality.

Within-read non-independence at HP positions. HP length errors are signal-level events affecting the entire run at once, not independent per-base errors [2] §3.4. The per-position q_{ctx} tabulated in the empirical analysis is a compact parameterization for cross-context comparison; the binomial model used here is correct because it operates across reads (independent draws from the sequencing process), not within a read. Practitioners should not interpret the per-position q_{ctx} for HP positions as a mechanistic rate.

Non-stationary error rates within a run. If error rates drift during a sequencing run (flow cell degradation, temperature variation), the binomial model’s identical-trials assumption is violated. Phase 2 per-run estimation partially mitigates this for between-run drift but cannot address within-run drift.

q_{ctx} provenance for novel basecaller versions. Shipped q_{ctx} tables are derived from empirical analysis of specific basecaller versions [2]. New basecaller releases require either re-characterization or use of the closest available table with explicit acknowledgment of the mismatch. Per-run estimation (Phase 2) addresses this but is not yet implemented.

Long HP runs ($L \geq 6$). The framework does not extend to $L \geq 6$ HP variants; these continue to be blanket-discarded as in the existing pipeline. The HP error analysis [2] §8.1 documents that accuracy degrades sharply at $L \geq 6$ and variant calling becomes unreliable; this is a property of the data, not the framework.

Biological variant priors. The framework treats all positions as having the same prior probability of carrying a real biological variant. In ITS, conserved regions (5.8S) and hypervariable regions (ITS1, ITS2) have very different prior variant densities. A Bayesian extension could incorporate position-specific priors; the current framework does not.

10 Conclusion

The CER framework [1] provides a transparent statistic for variant significance that is interpretable against platform error knowledge. Its extension to context-dependent error rates — demanded by the empirical characterization of HP errors [2] — requires changes to the inputs of the CER equation, not to its structure. The unified equation (Section 2) accommodates per-context error rates and mixed-context multi-position variants without abandoning the binomial framework.

The CER factor (Section 3) is the practical metric that recovers cross-variant comparability under context-aware q_{ctx} . It preserves the human-interpretable units that motivated the original framework, degrades linearly under realistic q_{ctx} uncertainty, and re-scales cleanly when the empirical error table is updated.

The implementation in speconsense (Section 5) integrates the framework with existing pipeline structure: the N denominator follows the conservative choice at each stage, identity groups define the pairwise comparison set, and HP variant detection is reordered so that $L \leq 5$ HP length differences become candidate variants subject to CER rather than being unconditionally discarded.

The metadata reproducibility design (Section 6) ensures that stored output can be reinterpreted under updated assumptions. As basecaller chemistry evolves and empirical error tables improve, no recomputation from raw reads is required — the pipeline’s conclusions remain adaptable to the moving target of nanopore error characterization.

Phase 2 work — per-run q_{ctx} estimation, an empirical-distribution model for HP variants that captures within-read signal-level dependencies, and incorporation of biological-variant priors — remains future work, scoped beyond the initial context-aware deployment.

Acknowledgments

The empirical case studies in this report use a sequencing run produced by Mycota Lab (Hoosier Mushroom Society) under the protocol of Russell [4]. The author thanks Stephen D. Russell for sharing the run-level outputs.

Substantial portions of the text were drafted with the assistance of Claude (Anthropic, Opus 4.7). The core ideas, framework design, and implementation choices were provided by the author; the AI contributed to exposition, L^AT_EX formatting, and iterative drafting under human direction. The author takes full responsibility for the content.

A Metadata JSON schema (full example)

A representative example of the schema-2.0 metadata emitted by the current implementation (condensed for exposition; the `parameters` block carries the full set of CLI arguments in practice). Two identity groups are shown: a populated group (`g1`) with an anchor and two non-anchor variants, and a singleton group (`g2`).

```
{
  "schema_version": "2.0",
  "version": "0.7.7",
  "sample_name": "ONT08.75-C10--iNat262724197",
  "timestamp": "2026-04-20T18:09:14.624236",

  "parameters": {
    "algorithm": "graph",
    "min_identity": 0.9,
    "min_size": 3,
    "max_sample_size": 100,
    "presample_size": 1000,
    "min_variant_frequency": 0.1,
    "min_variant_count": 3,
    "significance_level": 1e-05,
    "group_identity": 0.85,
    "error_model": "dorado-v5.0"
```

```

},
"input_file": "ONT08.75-C10--iNat262724197.fastq",
"total_input_reads": 1000,
"identity_groups": [
  {
    "group_id": "g1",
    "group_N": 974,
    "members": ["1.v1", "1.v2", "1.v3"]
  },
  {
    "group_id": "g2",
    "group_N": 7,
    "members": ["2.v1"]
  }
],
"variants": [
  {
    "cluster_id": "1.v1",
    "identity_group": "g1",
    "group_rank": 1, "variant_rank": 1,
    "M": 462, "N": 974,
    "K": null, "n_sites": null, "context_tags": null,
    "q_ctx_per_position": null,
    "compared_against_idx": null,
    "cer_factor": null, "cer_pstar": null,
    "err_factor": 0.303,
    "err_factor_obs_sum": 3.13,
    "err_factor_exp_sum": 10.33,
    "err_factor_cols": 781
  },
  {
    "cluster_id": "1.v2",
    "identity_group": "g1",
    "group_rank": 1, "variant_rank": 2,
    "M": 133, "N": 974, "K": 2, "n_sites": 560,
    "context_tags": ["non-hp-sub", "non-hp-sub"],
    "q_ctx_per_position": [0.0059, 0.0059],
    "compared_against_idx": 0,
    "cer_factor": 46.500, "cer_pstar": 0.274,
    "err_factor": 0.445,
    "err_factor_obs_sum": 4.61,
    "err_factor_exp_sum": 10.35,
    "err_factor_cols": 781
  },
  {
    "cluster_id": "1.v3",
    "identity_group": "g1",
    "group_rank": 1, "variant_rank": 3,
    "M": 49, "N": 974, "K": 1, "n_sites": 561,
    "context_tags": ["hp-l3-A-del1"],
    "q_ctx_per_position": [0.0097],

```

```

    "compared_against_idx": 0,
    "cer_factor": 2.085, "cer_pstar": 0.0202,
    "err_factor": 0.453,
    "err_factor_obs_sum": 4.67,
    "err_factor_exp_sum": 10.32,
    "err_factor_cols": 780
  }
]
}

```

B Default q_{ctx} table for Dorado v5.0

Drawn from [2] §8.6 (re-estimation under the current speconsense pipeline, 0.7.7). Values are filtered per-position error rates (after biological-variant filtering) for the ont98 dataset (R10.4.1, Dorado SUP v5.0.0). HP rates use the `v2_ef` primary-anchor values (approach 1), where mode-based ground truth is most reliable; non-HP rates use the all-cluster pooled values (approach 2), which match the operational distribution CER evaluates against in production. Per-base and per-direction sub-dimensions are pooled for the reasons given in Section 4.1.1: the effects are likely below the cross-run noise floor (not directly characterized in [2]) and cross-cell counts are too sparse to resolve the finer segmentation stably.

Table 2: Shipped q_{ctx} values for Dorado v5.0 (per-position error rate). This is the complete contents of `dorado-v5.0.yaml`; per-base and per-direction variation is folded into each entry.

Context tag	q_{ctx}	Source
non-hp-sub	0.0039	[2] §8.6 (approach 2, ont98)
non-hp-indel	0.0074	[2] §8.6 (approach 2, ont98)
hp-11	0.0047	[2] §8.6 (v2_ef, ont98)
hp-12	0.0062	[2] §8.6 (v2_ef, ont98)
hp-13	0.0076	[2] §8.6 (v2_ef, ont98)
hp-14	0.0081	[2] §8.6 (v2_ef, ont98)
hp-15	0.0087	[2] §8.6 (v2_ef, ont98)
hp-16+	N/A	blanket discard, [2] §8.1

Future refinement. Finer segmentation — per-base (**hp-15-G** vs **hp-15-A**, per [2] §7.6) or per-direction (**hp-13-A-del1** vs **hp-13-A-ins1**) — is straightforward to add by extending the YAML with additional keys. The lookup function already accepts any string key, and the context tag persisted in the metadata JSON records the full (b, L, δ) triple. Downstream recomputation under a finer table therefore requires no upstream re-run. The current simplification reflects the data available at publication time: with likely cross-run variation on the same basecaller (not characterized in [2]), finer cells would risk false precision rather than delivering additional discrimination. Per-base and per-direction re-estimation under the current pipeline has not been performed; the shipped pooled values in Table 2 are the authoritative production defaults.

HP-interface substitutions inherit the **non-hp-sub** rate automatically: the context classifier reports them as `NON_HP_SUB` tags per the finding in [2] §4 that interface substitution rates are indistinguishable from non-HP. No separate table entry is needed.

References

- [1] J. Walker. Critical error rate analysis for MSA-based variant phasing in nanopore amplicon sequencing. Technical report, 2026.
- [2] J. Walker. Homopolymer error rate analysis for nanopore amplicon sequencing. Technical report, 2026.
- [3] J. Walker. Speconsense: High-quality clustering and consensus generation for Oxford Nanopore amplicon reads. <https://github.com/joshuaowalker/speconsense>, 2024.
- [4] S. D. Russell. ONT DNA barcoding fungal amplicons w/ MinION & Flongle V.3. *protocols.io*, March 2023. <https://dx.doi.org/10.17504/protocols.io.36wgq7qyk5/v3>.
- [5] C. Delahaye and J. Nicolas. Sequencing DNA with nanopores: Troubles and biases. *PLoS ONE*, 16(10):e0257521, 2021.
- [6] M. Sereika, R. H. Kirkegaard, S. M. Karst, T. Y. Michaelsen, E. A. Sørensen, R. D. Wollenberg, and M. Albertsen. Oxford Nanopore R10.4 long-read sequencing enables the generation of near-finished bacterial genomes from pure cultures and metagenomes without short-read or reference polishing. *Nature Methods*, 19:482–488, 2022.
- [7] R. R. Wick, L. M. Judd, and K. E. Holt. Assembling the perfect bacterial genome using Oxford Nanopore and Illumina sequencing. *PLOS Computational Biology*, 19(3):e1010905, 2023.
- [8] C. Lee. Generating consensus sequences from partial order multiple sequence alignment graphs. *Bioinformatics*, 19(8):999–1008, 2003.
- [9] R. Vaser, I. Sović, N. Nagarajan, and M. Šikić. Fast and accurate de novo genome assembly from long uncorrected reads. *Genome Research*, 27(5):737–746, 2017.